Bayesian BIM Reconstruction from LiDAR Point Clouds — Research Brief

Condensed context document for development sessions · July 2026

1. Research Question

Can Bayesian inference recover structured architectural representations — specifically floor plans as planar graphs — from raw 3D LiDAR point cloud scans of buildings? The pipeline decomposes the full BIM reconstruction problem into four sequential layers: storey detection, floor plan inference, wall dimensioning, and opening detection. **Floor plan inference is the primary research focus** and the hardest subproblem.

2. Core Problem Formulation

Data

For a single building storey, the observed data is a collection of 2D point cloud slices at multiple heights:

```
D = { (h_i, P_i) }    where P_i is a 2D point cloud at height h_i
```

Multiple heights are used because different heights reveal different information — 0.3 m captures wall bases cleanly, 1.0 m captures the main wall surface with some furniture occlusion, 1.8 m is above most furniture. Aggregating across heights gives robustness to occlusion at any single level.

Representation

A floor plan is a planar graph G = (V, E) where nodes are wall segments (line segments) and connectivity is encoded via shared junction points. Rooms are **not** explicitly encoded — they emerge implicitly as cycles in the graph. This keeps the representation minimal and mathematically clean.

A junction is a 2D point (x, y) in metres where two or more wall segments meet. A wall segment is defined by references to two junction IDs — moving a junction moves all incident segments simultaneously. The graph is subject to a planarity constraint: segments may only intersect at shared junctions.

Posterior Target

```
P(G | D)  ∝  P(D | G) · P(G)
```

The likelihood P(D | G) and prior P(G) are described below. The inference problem is variable-dimension: the number of wall segments |V| is unknown, so the posterior is defined over a union of spaces of different dimensionality. This is the central computational challenge.

3. Likelihood Model

Analytical Likelihood

For each observed point p in slice P_i at height h_i, the likelihood is a mixture of a wall-distance Gaussian and a uniform clutter term:

```
P(p | G, h) = α(h) · N( d(p,G) ; 0, σ²(h) ) + (1−α(h)) · U
```

where d(p, G) is the distance from point p to the nearest wall segment, σ²(h) is a height-dependent variance, α(h) is a height-dependent mixture weight, and U is uniform over the bounding box. The height-dependent parameters encode that some heights are noisier than others. Points near walls have high likelihood; furniture and clutter points are absorbed by the uniform component.

The full log-likelihood assumes independence across points and slices:

```
log P(D | G) = Σ_i  Σ_{p ∈ P_i}  log P(p | G, h_i)
```

Simulator-Based Likelihood

The simulator implements the forward model f(G, S, η) → D, mapping a floor plan, scanner positions S, and noise parameters η to a synthetic point cloud. This implicit likelihood is used by ABC and pseudo-marginal MCMC via Chamfer distance comparison:

```
P(D_obs | G)  ≈  exp( −γ · ChamferDistance( f(G, S, η), D_obs ) )
```

4. Prior P(G)

The prior encodes architectural knowledge as a sum of log-penalty terms:

```
log P(G) = −λ_k·|V|  −  λ_ortho·Σ angle_penalty(■)
           −  λ_junc·Σ junction_penalty(j)
           −  ∞ · [any segment shorter than ■_min]
```

Prior Term	What It Encodes	Key Parameter
Complexity	Penalise unnecessary segments; prefer simpler graphs	λ_k
Orthogonality	Manhattan world — walls should meet at right angles	λ_ortho
Min length	Hard zero probability for very short segments	■_min = 0.1 m
Junction regularity	Prefer T- and L-junctions over arbitrary angles	λ_junc
Room size	Cycle areas should fall in plausible range for building type	μ_room, σ_room

5. The Simulator — Why It Is Central

The simulator is the most leveraged piece of the codebase. It serves five roles simultaneously, and the choice to build it is what unlocks the full range of inference approaches:

- **Likelihood evaluation** — implicitly defines P(D | G) for ABC and pseudo-marginal MCMC without requiring an analytic form
- **Training data generation** — produces unlimited (G, D) pairs for SBI neural network training, eliminating the paired data scarcity problem
- **Gradient source** — implemented differentiably in JAX, enabling gradient-based MAP inference via jax.grad through junction positions
- **Validation tool** — running the simulator on known floor plans and comparing to real scans quantifies the sim-to-real gap
- **Domain randomisation** — randomising noise parameters η per call makes learned likelihoods robust to varying scan quality

Key Design Decision — Hard vs Soft Occlusion

Hard occlusion (each ray returns only the nearest wall hit) is physically correct but involves an argmin that is not differentiable. Soft occlusion approximates this via a softmin, enabling gradient flow at the cost of physical exactness. Both modes are implemented and selected via a flag — hard occlusion for ABC and data generation, soft occlusion when gradients are required.

6. Inference Approaches

Six inference approaches are implemented behind a common interface. The simulator underpins most of them. They are ordered here from most to least principled:

Method	Mechanism	Posterior	Speed	Key Dependency
RJMCMC	Markov chain over variable-dim graphs; birth/death/perturbation/merge/split moves	Exact	Slow	Analytical likelihood
SMC	Particle population reweighted slice-by-slice; RJMCMC rejuvenation kernel	Exact	Parallel	Analytical likelihood
ABC-MCMC	Accept proposals where simulated D matches observed D within threshold ε	Approx	Slow	Simulator
Pseudo-marginal MCMC	Monte Carlo likelihood estimate inside MH acceptance ratio	Exact	Slow	Simulator
SBI / NPE	PointNet encoder + flow decoder trained on synthetic (G,D) pairs	Approx	Fast	Simulator + training data
MAP / Gradient	jax.grad through differentiable simulator; optimise junction positions	Point	Fast	Differentiable simulator

Recommended Development Order Within Inference

- **Prior + analytical likelihood** — validate that prior samples look like floor plans and likelihood is well-scaled
- **MAP inference** — fastest to implement; validates the likelihood gradient before committing to MCMC
- **RJMCMC** — reference Bayesian implementation; correctness of MH Jacobian terms is critical and error-prone
- **SMC** — reuses RJMCMC kernel as rejuvenation; adds parallelism and anytime approximation
- **ABC** — requires simulator; implement after simulator is validated against real scans
- **SBI** — requires large synthetic dataset; implement last when simulator is stable

7. The Variable-Dimension Challenge

The number of wall segments |V| is unknown. The posterior is therefore defined over a union of spaces of different dimensionality — standard MCMC and variational inference do not apply directly. This is the deepest technical challenge in the project.

RJMCMC Solution

Reversible-Jump MCMC handles transdimensional moves via carefully constructed proposals that maintain detailed balance across spaces of different dimension. The key moves are:

- **Birth/Death** — add or remove a wall segment; requires dimension-matching Jacobian in MH ratio
- **Junction perturbation** — move a junction position (updates all incident segments); standard MH
- **Junction merge/split** — collapse two nearby junctions into one or split one into two; transdimensional
- **Gibbs-within-RJMCMC** — sample junction positions via Gibbs given fixed topology; conditionally Gaussian under analytical likelihood, dramatically improves mixing

JAX Representation

JAX requires fixed-size arrays for JIT compilation and vmap. The FloorPlanGraph is exported as padded fixed-size arrays with boolean validity masks:

```
segments:       (K_max=128, 4)   float   — (x1,y1,x2,y2) per segment
segment_mask:   (K_max=128,)     bool    — True for active segments
junctions:      (J_max=256, 2)   float   — (x,y) per junction
junction_mask:  (J_max=256,)     bool    — True for active junctions
```

Mutation happens on the Python FloorPlanGraph object. JAX arrays are exported fresh when the simulator or inference code needs them. These exports are immutable.

8. Key Design Decisions and Their Rationale

Rooms as cycles, not explicit nodes

Encoding rooms explicitly would be redundant and would require defining room-level connectivity separately from wall connectivity. Rooms as cycles emerge from the graph structure naturally, keeping the representation minimal. Planarity is therefore a structural constraint — walls cannot cross — enforced by the prior and RJMCMC proposal design.

Simulator built in JAX from the start

JAX's jit, vmap, and grad are all needed simultaneously: jit for speed during SBI training (millions of simulator calls), vmap for batching over rays and scanner positions, grad for differentiable MAP inference. Retrofitting differentiability into a non-JAX simulator is very painful. The upfront cost is justified.

Likelihood injected into inference, never hardcoded

All inference algorithms receive a LikelihoodModel via a Protocol interface. Swapping the analytical Gaussian likelihood for the simulator-based implicit likelihood requires zero changes to inference code. This is what makes the codebase a research platform rather than a collection of scripts.

Junction-based endpoint snapping

Free-floating segment endpoints lead to near-misses where walls that should share a junction have endpoints at slightly different positions. This breaks cycle detection (rooms fail to close) and creates fragmented graphs. Snapping endpoints to a shared junction set solves this and naturally enforces planarity via the junction lattice.

Scanner positions as explicit model inputs

Scanner positions are always passed explicitly to the simulator — never hardcoded. In real surveys they are known from registration. In synthetic generation they are randomised for domain randomisation. In future work they could be treated as latent variables and inferred jointly with G.

9. The Sim-to-Real Gap

The simulator is only useful if it produces point clouds that resemble real scans. The gap between simulated and real data is the primary practical risk for SBI and domain randomisation approaches.

- **Primary metric:** Chamfer distance between simulated and real FloorPointCloud for a known floor plan
- **Calibration:** optimise NoiseModel parameters (range_sigma, angular_sigma, outlier_rate, missing_rate, height_sigma) to minimise Chamfer distance against real scans

• **Domain randomisation:** sample NoiseModel parameters from a prior distribution per simulator call during SBI training — produces a learned likelihood robust to scan quality variation

• **Do not begin SBI training until the simulator is calibrated** — the sim-to-real gap directly bounds how well a learned likelihood transfers to real data

10. Component Architecture

Four components with a strict one-way dependency order:

Component	Primary Responsibility	Key Files
representation/	FloorPlanGraph, Junction, WallSegment, Building, constraints, JAX export	primitives.py, floor_plan.py, constraints.py
simulator/	Ray casting, noise model, simulate(), calibration, differentiable mode	ray_caster.py, noise_model.py, engine.py
data/	Real scan ingestion (.las/.ply/.e57), preprocessing, synthetic wrappers	loaders.py, preprocessing.py, synthetic.py
inference/	Prior P(G), likelihood protocol, RJMCMC, SMC, ABC, SBI, MAP, refinement	prior.py, likelihood.py, result.py, rjmcmc/, sbi/, map/

Cross-Component Types

These types cross component boundaries and must remain consistent across all components:

Type	Defined In	Used By
FloorPlanGraph	representation/floor_plan.py	simulator, data, inference
FloorPointCloud	simulator/engine.py	data (produced), inference (consumed)
PointCloudSlice	simulator/engine.py	data (produced), inference (consumed)
NoiseModel	simulator/noise_model.py	inference (injected), data (DatasetGenerator)
InferenceResult	inference/result.py	experiments/, notebooks/

11. Critical Implementation Warnings

• **RJMCMC Jacobian terms:** errors in the MH acceptance ratio for transdimensional moves are silent — the chain runs without errors but targets the wrong distribution. Validate against known analytic posteriors before use on real data.

• **JAX PRNG discipline:** all stochastic operations must use explicit jax.random.PRNGKey. No numpy.random or Python random calls inside jit-compiled functions — this breaks reproducibility.

• **No Python loops over arrays in JAX:** use jax.vmap for batched operations over rays, segments, and particles. Python loops prevent JIT compilation and negate the performance benefits.

• **Do not start inference before validating the simulator:** debugging inference and simulator simultaneously is intractable. Validate simulator output against real scans first.

• **Units:** all coordinates in metres, all angles in radians. Mixed units are a silent failure mode — there is no runtime error, only wrong results.

• **FloorPointCloud format parity:** the Data component's loaders and the Simulator must produce identical FloorPointCloud objects. Inference code must never branch on whether data is real or synthetic.

This document is a condensed research brief for development context. Full component specifications are in docs/representation.md, docs/simulator.md, docs/data.md, and docs/inference.md. The repository structure and tooling conventions are documented in CLAUDE.md.